รายงานวิจัยฉบับสมบูรณ์: สถาปัตยกรรม Edge AI และ Stacking Ensemble สำหรับเสื้ออัจฉริยะ "The Invisible Hug" เพื่อพยากรณ์ภาวะวิกฤตทางอารมณ์ในเด็กออทิสติก

บริบททางสรีรวิทยาชีวการแพทย์และหลักการทำงานของระบบ

ภาวะวิกฤตทางอารมณ์ (Meltdown) ในเด็กที่มีภาวะออทิสติกสเปกตรัม (Autism Spectrum Disorder: ASD) มักมีจุดกำเนิดมาจากภาวะการรับรู้ทางประสาทสัมผัสที่มากเกินพอดี (Sensory Overload) หรือความเครียดสะสมที่ไม่สามารถจัดการได้ด้วยกลไกทางอารมณ์ตามปกติ ก่อนที่เหตุการณ์การระเบิดอารมณ์จะเกิดขึ้น ร่างกายของเด็กจะเข้าสู่ระยะที่เรียกว่า "Rumbling Stage" ซึ่งเป็นช่วงเวลาสั้นๆ ที่ระบบประสาทอัตโนมัติ (Autonomic Nervous System: ANS) เริ่มแสดงสัญญาณความผิดปกติทางสรีรวิทยา โดยปกติแล้วระยะเวลานี้จะเกิดขึ้นล่วงหน้าประมาณ 1 ถึง 3 นาที[1] การแทรกแซงในช่วงเวลาวิกฤตนี้มีประสิทธิภาพสูงที่สุดในการป้องกันพฤติกรรมรุนแรง ดังนั้น การพัฒนาระบบที่สามารถพยากรณ์และตรวจจับสัญญาณในระยะ Rumbling Stage จึงเป็นเป้าหมายหลักของการออกแบบเสื้ออัจฉริยะแบบสวมใส่บนระบบเครือข่ายอินเทอร์เน็ตของสรรพสิ่ง (IoT) ที่มีชื่อว่า "The Invisible Hug" ซึ่งมีจุดประสงค์เพื่อตอบสนองต่อสัญญาณดังกล่าวด้วยการทำงานของระบบบำบัดด้วยแรงกดลึก (Deep Touch Pressure: DTP) อัตโนมัติ[2]

การวัดสัญญาณทางสรีรวิทยาผ่านอุปกรณ์สวมใส่จำเป็นต้องบูรณาการข้อมูลจากหลายเซนเซอร์ (Multi-sensor Fusion) เพื่อประเมินการทำงานของระบบประสาทอัตโนมัติทั้งในส่วนซิมพาเทติก (Sympathetic) และพาราซิมพาเทติก (Parasympathetic) อย่างแม่นยำ[4] สถาปัตยกรรมของเสื้ออัจฉริยะนี้ถูกกำหนดให้ประมวลผลข้อมูลจากเซนเซอร์ 4 ประเภทหลัก ได้แก่ สัญญาณความต้านทานไฟฟ้าของผิวหนัง (Electrodermal Activity: EDA), อัตราการเต้นของหัวใจ (Heart Rate: HR), อุณหภูมิผิวหนัง (Skin Temperature), และหน่วยวัดแรงเฉื่อย 3 แกน (3-axis IMU) ซึ่งตรวจจับความเร่งของการเคลื่อนไหว[4]

เพื่อให้ระบบสวมใส่นี้สามารถทำงานแบบเวลาจริง (Real-time) บนอุปกรณ์ประมวลผลปลายทาง (Edge Computing) ภายใต้ข้อจำกัดทางสภาพแวดล้อมที่คาดเดาไม่ได้ ปัญหาทางวิศวกรรมที่สำคัญที่สุดที่ต้องแก้ไขประกอบด้วยสามมิติหลัก มิติแรกคือการปนเปื้อนของสัญญาณรบกวนจากการเคลื่อนไหว (Motion Artifacts) ซึ่งรุนแรงมากในกรณีที่เด็กมีพฤติกรรมกระสับกระส่าย มิติที่สองคือความแปรปรวนของข้อมูลระหว่างบุคคล (Inter-subject Variability) ที่ทำให้โมเดลแบบสากล (Global Model) ขาดความแม่นยำ และมิติที่สามคือข้อจำกัดทางทรัพยากรการประมวลผลบนไมโครคอนโทรลเลอร์ที่ไม่อนุญาตให้ใช้ระบบปฏิบัติการขนาดใหญ่ รายงานฉบับนี้นำเสนอระเบียบวิธีวิจัยและสถาปัตยกรรมแบบเจาะลึก เพื่อนำเสนอแนวทางการประยุกต์ใช้อัลกอริทึม Adaptive Filter สำหรับขจัดสัญญาณรบกวน การวิศวกรรมคุณลักษณะแบบปรับฐานพลวัต (Dynamic Baseline) และการพัฒนาสถาปัตยกรรม Stacking Ensemble Model ตลอดจนแนวปฏิบัติในการนำไปใช้งานจริงบนไมโครคอนโทรลเลอร์ ESP32 โดยอาศัยภาษา C++ ล้วน[8]

กลยุทธ์การประมวลผลสัญญาณและการกรองสัญญาณรบกวนเชิงรุก

สัญญาณทางสรีรวิทยาที่บันทึกผ่านอุปกรณ์สวมใส่มีความอ่อนไหวอย่างยิ่งต่อการปนเปื้อนจากสิ่งแวดล้อมทาง

กลศาสตร์ การเปลี่ยนตำแหน่งของเซนเซอร์เพียงระดับมิลลิเมตร หรือการเกร็งกล้ามเนื้อสามารถเหนี่ยวนำให้เกิดสัญญาณรบกวนที่มีแอมพลิจูดสูงกว่าสัญญาณทางชีวภาพที่แท้จริงหลายเท่าตัว การประยุกต์ใช้เทคนิคการกรองสัญญาณแบบปรับตัว (Adaptive Filtering) จึงเป็นรากฐานที่กำหนดความสำเร็จของสถาปัตยกรรมทั้งหมด[11]

การปรับแต่งสัญญาณ EDA และ HR ผ่านสถาปัตยกรรม Recursive Least Squares (RLS)

สัญญาณอัตราการเต้นของหัวใจ (HR) และความต้านทานไฟฟ้าของผิวหนัง (EDA) เป็นพารามิเตอร์เชิงแสงและเชิงไฟฟ้าที่มีปฏิสัมพันธ์สูงกับการเคลื่อนไหวร่างกาย เมื่อพิจารณาถึงพฤติกรรมทางกายภาพของกลุ่มเป้าหมายที่มีแนวโน้มแสดงพฤติกรรมซ้ำๆ (Stimming) การประยุกต์ใช้ฟิลเตอร์เชิงเส้นแบบดั้งเดิมมักล้มเหลวเนื่องจากคลื่นความถี่ของการเคลื่อนไหวทับซ้อนกับคลื่นความถี่ของสัญญาณสรีรวิทยา ด้วยเหตุนี้ สถาปัตยกรรม Recursive Least Squares (RLS) จึงถูกกำหนดให้เป็นกลไกหลักในการทำความสะอาดข้อมูล โดยอาศัยข้อมูลเชิงเรขาคณิตจากการเคลื่อนไหวผ่านเซนเซอร์ IMU เป็นสัญญาณอ้างอิงของสัญญาณรบกวน (Noise Reference Signal)[13]
อัลกอริทึม RLS มีความได้เปรียบทางวิศวกรรมเหนืออัลกอริทึม Least Mean Squares (LMS) ในด้านอัตราการลู่เข้าหาคำตอบ (Convergence Rate) ที่รวดเร็วกว่า และความสามารถในการรักษาสมรรถนะแม้ในสภาวะที่อินพุตมีความสัมพันธ์ทางโครงสร้างสูง (Highly Correlated Input) การทำงานของ RLS มุ่งเน้นไปที่การลดผลรวมของกำลังสองของความคลาดเคลื่อนแบบถ่วงน้ำหนัก (Exponentially Weighted Least Squares) ซึ่งตอบสนองต่อการเปลี่ยนแปลงทางพลวัตของการเคลื่อนไหวได้ในระดับมิลลิวินาที[15]
กระบวนการคำนวณของ RLS Adaptive Filter ดำเนินการผ่านสมการเวกเตอร์และเมทริกซ์ที่ปรับปรุงสถานะอย่างต่อเนื่องตามรอบการทำงาน การประมาณการสัญญาณรบกวน $y(n)$ อาศัยอาร์เรย์ของข้อมูลอ้างอิงจากเซนเซอร์ IMU $u(n)$ ซึ่งคำนวณผ่านค่านอร์ม L2 (Signal Vector Magnitude) ของความเร่งทั้งสามแกน ผนวกเข้ากับเวกเตอร์น้ำหนักของฟิลเตอร์ $w(n-1)$ ที่ปรับปรุงมาจากรอบก่อนหน้า สัญญาณคลาดเคลื่อน $e(n)$ ซึ่งได้จากการหักล้างสัญญาณดิบจากเซนเซอร์ EDA หรือ HR $d(n)$ ด้วยสัญญาณรบกวนที่ประมาณค่าขึ้น จะทำหน้าที่เป็นผลลัพธ์บริสุทธิ์ของกระบวนการนี้และเป็นฟีดแบคสำหรับการปรับน้ำหนัก[17] กลไกการปรับปรุงน้ำหนักเกี่ยวข้องกับการคำนวณ Kalman Gain $k(n)$ และการปรับเมทริกซ์ความแปรปรวนร่วม $P(n)$ โดยมีพารามิเตอร์ที่สำคัญที่สุดคือสัมประสิทธิ์การลืม (Forgetting Factor, λ) ค่า λ กำหนดอัตราการสลายตัวของข้อมูลในอดีต ซึ่งสำหรับการเคลื่อนไหวของมนุษย์ที่มีลักษณะฉับพลัน (Burst Motion) การกำหนดค่า λ ให้อยู่ในช่วง 0.990 ถึง 0.999 เป็นจุดสมดุลที่ดีที่สุดระหว่างเสถียรภาพและความเร็วในการปรับตัวของไมโครคอนโทรลเลอร์[18]

การออกแบบฟิลเตอร์สำหรับสัญญาณอุณหภูมิผิวหนัง

แตกต่างจากสัญญาณไฟฟ้าหรือสัญญาณที่ได้จากแสง (Photoplethysmography) อุณหภูมิผิวหนังมีการเปลี่ยนแปลงทางอุณหพลศาสตร์ที่เชื่องช้ามาก (Low-frequency Dynamics) การเปลี่ยนแปลงของอุณหภูมิซึ่งเป็นผลมาจากการหดตัวของหลอดเลือดส่วนปลาย (Peripheral Vasoconstriction) ในระหว่างเกิดภาวะเครียดมักใช้เวลาหลายนาทีจึงจะแสดงผลได้อย่างชัดเจน[3]
ความเชื่องช้านี้ทำให้ปัญหา Motion Artifacts กลายเป็นเรื่องรอง ในขณะที่สัญญาณรบกวนหลักมักเกิดจากอุณหภูมิสิ่งแวดล้อม (Environmental Thermal Noise) หรือการสูญเสียการสัมผัสชั่วคราวระหว่างเซนเซอร์และผิวหนัง ด้วยเหตุนี้ อัลกอริทึม RLS จึงไม่มีความจำเป็นและเป็นการสูญเสียทรัพยากรการคำนวณโดยใช่เหตุ ระบบจึงประยุกต์ใช้ฟิลเตอร์มาตรฐานอย่าง Low-Pass Filter หรือกระบวนการคำนวณค่าเฉลี่ยเคลื่อนที่แบบง่าย

(Simple Moving Average: SMA) ที่หน้าต่างเวลา 10 วินาที เพื่อสร้างเสถียรภาพให้กับเส้นโค้งความร้อนและลดผลกระทบจากการอ่านค่าที่ผิดพลาดชั่วขณะ[7]

การวิศวกรรมคุณลักษณะและกลยุทธ์กรอบเวลาเชิงพื้นที่ (Temporal Windowing Strategy)

การพัฒนาแบบจำลองบนพื้นที่ทรัพยากรจำกัด (TinyML) ไม่สามารถใช้เทคนิค Deep Learning ที่ป้อนข้อมูลดิบ (Raw Data) เข้าสู่โครงข่ายประสาทเทียมโดยตรงได้ ความแม่นยำจึงขึ้นอยู่กับศาสตร์ของการสกัดคุณลักษณะ (Feature Extraction) ที่ทรงพลังภายใต้โครงสร้างหน้าต่างข้อมูลที่ออกแบบมาอย่างรอบคอบ[1]

กลยุทธ์การจัดหน้าต่างข้อมูลเพื่อพยากรณ์ภาวะวิกฤต (Sliding Window over 3-Minute History)

เพื่อพยากรณ์เหตุการณ์ Rumbling Stage ล่วงหน้า 1 นาที สถาปัตยกรรมข้อมูลถูกจัดแบ่งตามแกนเวลาโดยอาศัยกรอบเวลาสกัดข้อมูลแบบเลื่อนที่ (Sliding Window) ขนาด 15 วินาที การเลือกขนาด 15 วินาทีตั้งอยู่บนรากฐานทางสรีรวิทยาของการตอบสนองของความต้านทานผิวหนังแบบชั่วคราว (Phasic Skin Conductance Response) และช่วงเวลาที่สั้นที่สุดที่สามารถวิเคราะห์การแปรผันของอัตราการเต้นของหัวใจแบบสั้นพิเศษ (Ultra-short-term HRV) ได้อย่างมีนัยสำคัญ[20]

ระบบถูกตั้งค่าให้มองย้อนกลับไปเก็บประวัติศาสตร์ความยาว 3 นาที (180 วินาที) ทำให้ในหน่วยความจำจะต้องรักษาข้อมูลสถานะของคุณลักษณะที่ถูกสกัดแล้วจำนวน 12 กรอบเวลา (180/15 = 12 Windows) โครงสร้างข้อมูลเช่นนี้ช่วยให้โมเดลสามารถพิจารณาทั้งค่าสัมบูรณ์ภายในระยะเวลาสั้นๆ และความเปลี่ยนแปลง (Derivatives) เชิงโมเมนตัมตลอดช่วงเวลา 3 นาทีที่ผ่านมา เพื่อทำนายฉลากความเสี่ยงในวินาทีที่ 181 ถึง 240 ว่าจะเกินเกณฑ์วิกฤตความเครียดหรือไม่[23]

การสกัดคุณลักษณะน้ำหนักเบาเชิงสถิติ (Lightweight Statistical Feature Selection)

การออกแบบเพื่อ ESP32 บังคับให้หลีกเลี่ยงกระบวนการคำนวณที่ซับซ้อนในโดเมนความถี่ (Frequency-Domain) เช่น Fast Fourier Transform (FFT) ซึ่งกินทรัพยากรแรมและรอบประมวลผล คุณลักษณะที่เลือกใช้จึงถูกจำกัดอยู่เพียงโดเมนเวลา (Time-Domain) ที่พิสูจน์แล้วว่ามีนัยสำคัญทางพยาธิสรีรวิทยาของภาวะเครียด ดังแสดงตารางต่อไปนี้[5]

เซนเซอร์ทางสรีรวิทยา	คุณลักษณะที่ถูกเลือกใช้งานต่อกรอบ 15 วินาที (Lightweight Time-Domain Features)	คำอธิบายและความเชื่อมโยงกับระบบประสาท
EDA (ความต้านทานผิวหนัง)	ค่าเฉลี่ยระดับความต้านทานพื้นฐาน (Mean SCL), ความถี่ของการเกิดปฏิกิริยาเฉียบพลัน (SCR Frequency), และแอมพลิจูดเฉลี่ยของจุดสูงสุด (Mean SCR Amplitude)	สะท้อนระดับความตื่นตัว (Arousal) ที่สั่งการโดยระบบซิมพาเทติก การเพิ่มขึ้นของ SCR Frequency บ่งชี้ถึงสภาวะที่ผู้ป่วยกำลังสูญเสียการควบคุมทางอารมณ์ การนับจำนวนยอดคลื่น

		(Peak Detection) สามารถคำนวณด้วยลอจิกพื้นฐานได้รวดเร็ว[4]
HR (อัตราการเต้นหัวใจ)	ค่าเฉลี่ยความถี่ของการเต้นหัวใจ (Mean HR), ค่ารากที่สองของค่าเฉลี่ยความแตกต่างของช่วงเวลาที่อยู่ติดกัน (RMSSD), และส่วนเบี่ยงเบนมาตรฐานของช่วงเวลา (SDNN)	RMSSD และ SDNN เป็นมาตรวัดสำคัญสำหรับ Ultra-short-term HRV ที่ 15 วินาที สะท้อนการถดถอยของการทำงานในระบบพาราซิมพาเทติก (Parasympathetic Withdrawal) เมื่อเกิดความเครียดรุนแรง[5]
Skin Temperature (อุณหภูมิ)	ค่าเฉลี่ยอุณหภูมิ (Mean Temperature) และความชันของแนวโน้มอุณหภูมิ (Temperature Slope)	การหดตัวของเส้นเลือดส่วนปลายทำให้อุณหภูมิพื้นผิวลดลง ความชันบ่งบอกถึงทิศทางของการเปลี่ยนแปลง ซึ่งมีความน่าเชื่อถือกว่าการพิจารณาค่าอุณหภูมิเพียงอย่างเดียว[3]
IMU (หน่วยวัดแรงเฉื่อย)	ค่าเฉลี่ยของขนาดเวกเตอร์สัญญาณ (SVM Mean) และความแปรปรวน (Variance of SVM)	ข้อมูลนี้ตรวจจับการเพิ่มขึ้นของพฤติกรรมกระสับกระส่าย (Fidgeting) หรือพฤติกรรมการขยับซ้ำๆ (Stimming) ซึ่งพบมากในกลุ่มออทิสติกช่วง Rumbling Stage[9]

กลยุทธ์การแก้ปัญหาความแตกต่างระหว่างบุคคลด้วย Normalization แบบเลื่อน

จุดอ่อนระดับวิกฤตของแบบจำลองสรีรวิทยาคือความแตกต่างทางร่างกายระหว่างตัวบุคคล (Inter-subject Variability) ข้อมูลพื้นฐานของการเต้นหัวใจและระดับเหงื่อของเด็กสองคนมีความแตกต่างกันอย่างสิ้นเชิง การสร้างโมเดลร่วม (Global Model) ที่พึ่งพาค่าดิบจึงหลีกเลี่ยงไม่ได้ที่จะนำไปสู่ความผิดพลาดในการพยากรณ์[1] แนวทางแก้ปัญหานี้ดำเนินการผ่านกลยุทธ์ "การปรับบรรทัดฐานรายบุคคล" (Subject-wise Baseline Normalization) ระบบถูกตั้งโปรแกรมให้บันทึกสถานะพื้นฐานในช่วง 5 นาทีแรกขณะที่ผู้ใช้งานอยู่ในสภาวะสงบ (Resting Baseline) จากนั้นคุณลักษณะทางสถิติทั้งหมดที่สกัดได้จากกรอบเวลา 15 วินาที จะถูกแปลงให้อยู่ในรูปของร้อยละความผิดพลาดสัมพันธ์ (Relative Percentage Error) เมื่อเทียบกับฐาน ดังสมการ:

$$\text{Relative Feature} = \left(\frac{\text{Current Value} - \text{Baseline}}{\text{Baseline}} \right) \times 100$$

การคำนวณเชิงสัมพันธ์นี้บังคับให้โมเดลปัญญาประดิษฐ์ให้ความสนใจเฉพาะ "อัตราและการเร่งของการเปลี่ยนแปลง" (Magnitude of Shift) มากกว่าจดจำค่าสถิติสัมบูรณ์ ซึ่งเพิ่มศักยภาพของการวางนัยทั่วไป (Domain Generalization) ไปยังกลุ่มผู้ใช้ที่ไม่เคยมีข้อมูลมาก่อน[1]

ยิ่งไปกว่านั้น สัญญาณทางสรีรวิทยามีปรากฏการณ์ลอยตัว (Baseline Drift) เนื่องจากการเปลี่ยนแปลงอุณหพลศาสตร์ของร่างกายตามช่วงเวลาของวัน การใช้ค่าบรรทัดฐาน 5 นาทีแรกตลอดทั้งวันจะก่อให้เกิดความคลาดเคลื่อนสะสม ระบบจึงบูรณาการ "ความเฉลี่ยเคลื่อนที่แบบอิงเลขชี้กำลัง" (Exponential Moving Average: EMA) เพื่อปรับแก้ค่า Baseline อย่างต่อเนื่องและราบรื่นโดยใช้หน่วยความจำระดับคงที่ (O(1))[28] สูตรการอัปเดต EMA ดำเนินการเฉพาะเมื่อระบบประเมินว่าผู้ใช้อยู่ในสภาวะสงบ (Non-stress):

$$EMA_t = \alpha \cdot X_t + (1 - \alpha) \cdot EMA_{t-1}$$

โดย α คือปัจจัยการเรียบเนียน (Smoothing Factor) ที่มีค่าต่ำมาก (เช่น 0.01 ถึง 0.05) เพื่อให้บรรทัดฐานซึมซับเฉพาะการเบี่ยงเบนระยะยาว (Long-term Drift) อย่างช้าๆ และปฏิเสธการแกว่งตัวฉับพลันที่อาจเกิดจากปัจจัยความเครียดระยะสั้น[31]

สถาปัตยกรรมแบบจำลอง Stacking Ensemble สำหรับ Edge Computing

ภายใต้ข้อจำกัดของไมโครคอนโทรลเลอร์ ESP32 การใช้โครงข่ายประสาทเทียมระดับลึก (Deep Neural Networks) แม้จะปรับแต่งด้วย TFLite Micro แล้ว ก็ยังบริโภคหน่วยความจำมหาศาลและควบคุมพฤติกรรมได้ยาก สถาปัตยกรรม Stacking Ensemble (Stacked Generalization) จึงถูกเลือกใช้เพื่อลดทอนความซับซ้อน ในขณะที่รักษาประสิทธิภาพการแบ่งแยกข้อมูลระดับสูงไว้ โดยใช้หลักการรวมกลุ่มแบบจำลองที่มีอคติแตกต่างกัน (Heterogeneous Base Models) เพื่อแก้ไขจุดอ่อนซึ่งกันและกัน[34]

การจำแนกความเชี่ยวชาญระดับฐาน (Level-0 Base Learners)

การแบ่งแยก Base Learners อย่างมีกลยุทธ์กระทำโดยการมอบหมายฟีเจอร์จากเซนเซอร์คนละประเภทให้โมเดลคนละตัวรับผิดชอบ (Sensor-specific Experts) ซึ่งเพิ่มความทนทานต่อกรณีที่เซนเซอร์บางตัวให้ข้อมูลที่ผิดปกติ[37]

1. **Random Forest Classifier (RF)**: โมเดลนี้ถูกกำหนดให้ทำหน้าที่ผู้เชี่ยวชาญด้านสัญญาณระบบประสาท (EDA และ HR) โครงสร้างต้นไม้การตัดสินใจแบบเอนเซมเบิลภายใน RF มีความเหมาะสมอย่างยิ่งต่อการเรียนรู้ปฏิสัมพันธ์แบบไม่เป็นเชิงเส้น (Non-linear Interactions) ที่ซับซ้อนของการแปรผันอารมณ์ ตลอดจนมีความทนทานต่อข้อมูลสุดโต่ง (Outliers) ที่เล็ดลอดมาจากกระบวนการฟิลเตอร์[27]
2. **Support Vector Machine (Linear SVM) หรือ Logistic Regression**: โมเดลนี้ทำหน้าที่เป็นผู้เชี่ยวชาญด้านพฤติกรรมทางกายภาพ (IMU และ Skin Temperature) ข้อมูลการเคลื่อนไหวและอุณหภูมิผิวหนังมักมีความสัมพันธ์เชิงเส้นที่ชัดเจนกว่า การใช้อัลกอริทึมเส้นขอบ (Hyperplane) แบบ Linear SVM ช่วยจำกัดจำนวนพารามิเตอร์ของโมเดลให้เหลือน้อยที่สุด ลดภาระหน่วยประมวลผลขณะสืบค้นอนุมานข้อมูล[9]

อัลกอริทึมผสานระดับสูง (Level-1 Meta-Learner)

ข้อมูลพยากรณ์จากโมเดล Base Learners ซึ่งอยู่ในรูปของความน่าจะเป็น (Probability Estimates) จะถูกป้อนเข้าสู่ Meta-Learner เพื่อเรียนรู้ว่าควรให้ความเชื่อมั่นต่อสัญญาณเตือนจากโมเดลใดในสภาวะใด[37]
อัลกอริทึม **Logistic Regression** ได้รับการประเมินให้เป็น Meta-Learner ที่เหมาะสมที่สุด โครงสร้างทางคณิตศาสตร์ที่เรียบง่ายซึ่งเป็นเพียงการคูณเมทริกซ์และส่งผ่านฟังก์ชันซิกมอยด์ (Sigmoid Function) ช่วยปกป้องสถาปัตยกรรมระดับเมตาจากการเกิดสภาวะเรียนรู้เกิน (Overfitting) ของข้อมูลความน่าจะเป็นที่มาจากระดับก่อนหน้า ยิ่งไปกว่านั้น Logistic Regression สามารถเปลี่ยนเป็นรหัส C++ สำหรับ ESP32 ได้โดยไร้ความ

จำเป็นต้องพึ่งพาไลบรารีขนาดใหญ่อย่างสมบูรณ์ ซึ่งตรงตามโจทย์ข้อจำกัดทรัพยากรทุกประการ[34]

ท่อส่งการฝึกสอนโมเดลและระเบียบวิธีในภาษา Python (Python Training Pipeline)

กระบวนการสร้างแบบจำลองเริ่มจากสภาพแวดล้อมภาษา Python อาศัยขีดความสามารถของ Scikit-learn แต่จำเป็นต้องมีการจัดการและจัดระเบียบข้อมูลตามอนุกรมเวลาที่รัดกุมเพื่อจำลองสถานการณ์กรอบเวลาแบบเลื่อนที่ (Temporal Sliding Window) ได้อย่างถูกต้อง

ประเด็นที่วิกฤตที่สุดของการพัฒนาแบบจำลองสรีรวิทยาข้ามบุคคลคือ ปรากฏการณ์ข้อมูลรั่วไหล (Data Leakage) หากกระบวนการ Cross-validation แยกชุดข้อมูลฝึกสอนและชุดทดสอบแบบสุ่มโดยไม่คำนึงถึงตัวบุคคล ข้อมูลจากผู้ใช้รายเดียวกันในกรอบเวลาต่างกันจะตกไปอยู่ทั้งในชุดฝึกสอนและชุดทดสอบ ส่งผลให้แบบจำลองจดจำลักษณะเฉพาะของบุคคลแทนที่จะเรียนรู้สัญญาณความเครียด การประเมินผลจะแสดงความแม่นยำสูงเกินจริง และล้มเหลวทันทีเมื่อนำไปใช้กับผู้ป่วยรายใหม่[42]

เพื่อแก้ปัญหานี้ ท่อส่งข้อมูลจำเป็นต้องประยุกต์ใช้เทคนิค **GroupKFold Cross-Validation** โดยระบุให้รหัสประจำตัวผู้ใช้ (Subject ID) เป็นตัวแปร groups การทำเช่นนี้บังคับให้อัลกอริทึมตัดแบ่งกลุ่มบุคคลอย่างชัดเจน เพื่อให้มั่นใจว่าในระหว่างกระบวนการตรวจสอบไขว้ ข้อมูลของผู้ใช้ที่อยู่ในชุดทดสอบ (Validation Fold) จะไม่เคยปรากฏในชุดฝึกสอน (Training Folds) มาก่อนแต่อย่างใด[43]

กระบวนการจำลองท่อส่งฝึกฝนบน Python (Python Pipeline) ถูกอธิบายทางลอจิกดังต่อไปนี้:

1. **Dataset Structuring**: กระบวนการจัดเรียงหน้าต่างเวลา ดำเนินการจัดโครงสร้าง 2D Array เพื่อบรรจุหน้าต่าง 12 เฟรมประวัติศาสตร์ (แต่ละเฟรม 15 วินาที รวม 3 นาที) ไว้ในแถวข้อมูล (Row) เดียวกันสำหรับการพยากรณ์เป้าหมายฉลากระดับความเครียด (Label Y) ที่อ้างอิงจากนาทีที่ 4 ของเหตุการณ์[23]
2. **Level-0 Definition**: การกำหนดกลุ่ม Base Learners ร่วมกับการทำ Scaling พื้นฐาน เช่น การใช้ StandardScaler ใน Pipeline ก่อนเข้า SVM[36]
3. **Out-of-Fold Generation**: การเตรียมชุดข้อมูลสำหรับ Level-1 ของ Stacking ทรงประสิทธิภาพด้วยกระบวนการสร้างการทายผลนอกกลุ่มพับ (Out-of-Fold Predictions) ซึ่งจัดการได้อัตโนมัติผ่าน StackingClassifier ของ Scikit-learn พร้อมการกำหนดพารามิเตอร์ cv=GroupKFold(n_splits=5) [cite: 35, 38, 46]
4. **Ensemble Finalization**: ตัวประเมินผลตัวสุดท้าย (Final Estimator) ในที่นี้คือ Logistic Regression จะได้รับการปรับน้ำหนักโดยอาศัย Output Probabilities ที่มาจากการทดสอบไขว้ในขั้นตอนก่อนหน้า กระบวนการนี้ทำให้ Meta-Learner เรียนรู้ถึงจุดอ่อนและแนวโน้มความผิดพลาด (Bias) ของ Base Learner แต่ละตัวอย่างเป็นระบบ[36]

การถ่ายโอนเทคโนโลยีสู่สภาพแวดล้อมประมวลผลปลายทาง (Edge Deployment on ESP32)

เมื่อแบบจำลองเรียนรู้และสอบเทียบสำเร็จใน Python กระบวนการสุดท้ายคือการแปลงสถานะของแบบจำลองให้สามารถทำงานประสานกับฮาร์ดแวร์ ESP32 ด้วยระดับความหน่วงทางเวลา (Latency) ที่ต่ำที่สุด การหลีกเลี่ยงระบบปฏิบัติการแบบรวมศูนย์หรือไลบรารีขนาดใหญ่เป็นแนวทางที่มีเสถียรภาพที่สุดในบริบทของอุปกรณ์สวมใส่ทางการแพทย์แบบเรียลไทม์[8]

การปฏิรูปโมเดลด้วย emlearn สู่ไลบรารีระดับล่าง

แทนที่จะนำแบบจำลองไปเข้าสู่ท่อส่ง TensorFlow Lite ที่ต้องมีการจองพื้นที่ประมวลผล (Memory Arena Allocator) สำหรับ Interpreter ซึ่งมักใช้พื้นที่เกิน 20 KB ระบบเลือกสกัดความซับซ้อนของ Stacking ผ่าน

เครื่องมือ **emlearn** นวัตกรรมนี้สามารถแปลคำสั่งของ Scikit-learn อย่าง Random Forest และ SVM ให้เป็นโครงสร้างข้อมูล struct แบบจัดเก็บล่วงหน้า และฟังก์ชันการตัดสินใจในภาษา C99 บริสุทธิ์[9] กระบวนการนี้ทำให้อัลกอริทึมปัญญาประดิษฐ์ไร้รอยต่อจากการใช้หน่วยความจำแบบพลวัต (No Dynamic Memory Allocations หรือ malloc) และมีขนาดไฟล์ไบนารีเพียงไม่กี่กิโลไบต์ ประทับลงพื้นที่ ROM โดยตรง ทำให้อัตราความเร็วในการประมวลผลสอดคล้องกับพารามิเตอร์ระดับ 10-20 มิลลิวินาที[8]

สถาปัตยกรรมโปรแกรมบน ESP32 และโครงสร้างตัวกันชนแบบวงแหวน

กระบวนการประมวลผลจริงถูกบริหารจัดการผ่านสถาปัตยกรรมการหน่วงเวลาและตัวกันชนแบบวงแหวน (Ring Buffer) บนหน่วยความจำ SRAM โครงสร้างการโปรแกรมถูกแบ่งภาคออกเป็นลำดับงานที่มีความต่อเนื่องดังนี้:

1. **Interrupt Data Acquisition**: นาฬิกาจับเวลาของ ESP32 บังคับใช้การสุ่มสัญญาณด้วยอัตรา 50 Hz สำหรับเซนเซอร์ IMU และ EDA นำเข้าข้อมูลบรรจุใน Array ชั่วคราว[8]
2. **Inline Adaptive Filtering**: ทุกครั้งที่ได้รับข้อมูล ฟังก์ชันสมการ RLS แบบ C++ ภายในจะอัปเดตเมทริกซ์ P เพื่อกรองทิ้งเศษซากของสัญญาณ Motion Artifact ทันที กระบวนการนี้กินระยะเวลาเพียงไม่กี่พันรอบสัญญาณนาฬิกา (Clock Cycles)[15]
3. **Feature Extraction and Circular Buffering**: เมื่อครบกรอบเวลา 15 วินาที ตัวแปรสถิติจะถูกสกัดตามตารางคุณลักษณะ การคำนวณ Variance หรือ Mean จะกระทำใน O(N) ตัวแปรสถิติกรอบนี้จะเข้าสู่โครงสร้าง Array แบบ Ring Buffer ที่ถูกตั้งค่าขนาดไว้ที่ 12 ตัวแปรตัวเก่าที่สุดจะถูกเขียนทับด้วยข้อมูลล่าสุดโดยไม่ต้องคัดลอกหน่วยความจำขยับตำแหน่ง (Memory Shifting) ซึ่งลดภาระทาง I/O ลงอย่างมหาศาล[9]
4. **Real-time Baseline Normalization**: ฟังก์ชัน C++ จะเรียกค่าล่าสุดไปเปรียบเทียบกับตัวแปรคงที่ dynamic_ema_baseline สัดส่วนเปอร์เซ็นต์ความผิดพลาดที่คำนวณได้จะทำหน้าที่เป็นข้อมูลป้อนเข้าตัวสุดท้าย
5. **Hierarchical Stacking Inference**: การทายผลบน Edge จะเริ่มด้วยการส่งกลุ่มฟีเจอร์ EDA/HR เข้าฟังก์ชันต้นไม้สุ่ม (RF) และส่งฟีเจอร์ IMU/Temp เข้าสู่ฟังก์ชันเส้นขอบ (SVM) ที่คอมไพล์ด้วย emlearn ผลสรุปที่เป็นเลขทศนิยม (Floats) สองตัวจะถูกนำไปคูณเชิงเมทริกซ์ (Matrix Multiplication) กับค่าน้ำหนักสัมประสิทธิ์ของ Logistic Regression ที่คอมไพล์ไว้ล่วงหน้า
6. **Actuator Protocol**: หากค่าสุดท้ายผ่านฟังก์ชัน Sigmoid แล้วเกินเกณฑ์วิกฤตความน่าจะเป็น (เช่น ≥ 0.80 เพื่อป้องกัน False Positives) คำสั่งส่งออกสัญญาณตรรกะระดับสูงผ่านพอร์ต GPIO จะกระตุ้นวงจรควบคุมมอเตอร์ปั๊มลม เพื่อเปิดใช้งานการบีบรัดบำบัดแบบ DTP แก่ผู้ใช้อัตโนมัติในสภาวะ Rumbling Stage ก่อนลุกลามเป็นพฤติกรรมระเบิดอารมณ์[2]

การบูรณาการกลไกสัญญาณทั้งหมดเข้าด้วยกันภายใต้กรอบข้อกำหนดด้านไมโครอิเล็กทรอนิกส์ สะท้อนถึงสมรรถนะสูงสุดของการนำสถาปัตยกรรม Edge AI ขนาดเล็กที่แม่นยำมาสู่เสื้ออัจฉริยะแบบสวมใส่ ซึ่งไม่เพียงนำเสนอความรัดกุมทางการประมวลผลสรีรวิทยา แต่ยังเป็นการพัฒนาแนวปฏิบัติสำคัญสำหรับแพลตฟอร์มการวิเคราะห์ทางการแพทย์อัจฉริยะแห่งอนาคต.

ผลงานที่อ้างอิง

1. Human Heterogeneity Invariant Stress Sensing - NSF PAR, https://par.nsf.gov/servlets/purl/10657055
2. Example of a Skin Conductance Response (SCR) curve. The graph... - ResearchGate, https://www.researchgate.net/figure/Example-of-a-Skin-Conductance-Response

-SCR-curve-The-graph-illustrates-three-typical_fig3_400620807
3. CalmSense: An IoT-Based Real-Time Stress and Anxiety Detection and Relief System - JETIR.org, https://www.jetir.org/papers/JETIR2511282.pdf
4. Electrodermal Activity (EDA): Guide & Top Wearables - Thryve Health, https://www.thryve.health/blog/electrodermal-activity-temperature-wearables
5. Ultra-short term HRV features as surrogates of short term HRV: a case study on mental stress detection in real life - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC6335694/
6. Hybrid Feature Learning for Wearable Stress Detection: Combining Domain Knowledge with Supervised Deep Learning - MDPI, https://www.mdpi.com/1424-8220/26/11/3451
7. MsWH: A Multi-Sensory Hardware Platform for Capturing and Analyzing Physiological Emotional Signals - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC9371105/
8. TinyML: Machine Learning on ESP32 with MicroPython - DEV Community, https://dev.to/tkeyo/tinyml-machine-learning-on-esp32-with-micropython-38a6
9. TinyML at the Edge: 5 Frameworks Powering IoT | by Nexumo - Medium, https://medium.com/@Nexumo_/tinyml-at-the-edge-5-frameworks-powering-iot-5c19a78e5211
10. Respiratory Monitoring in Motion: An Overview of Wearable Methods and Algorithmic Approaches for Reliable Assessment - MDPI, https://www.mdpi.com/2079-6374/16/6/306
11. Detection and Removal of Motion Artifacts in PPG Signals - ResearchGate, https://www.researchgate.net/publication/335056984_Detection_and_Removal_of_Motion_Artifacts_in_PPG_Signals
12. Cascade and parallel combination (CPC) of adaptive filters for estimating heart rate during intensive physical exercise from photoplethysmographic signal - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC5830890/
13. A Novel Adaptive Recursive Least Squares Filter to Remove the Motion Artifact in Seismocardiography - ResearchGate, https://www.researchgate.net/publication/339925280_A_Novel_Adaptive_Recursive_Least_Squares_Filter_to_Remove_the_Motion_Artifact_in_Seismocardiography
14. Adaptive Estimation Algorithm for Photoplethysmographic Heart Rate Based on Finite State Machine - MDPI, https://www.mdpi.com/2076-3417/14/24/11631
15. Recursive Least Square Algorithm - GeeksforGeeks, https://www.geeksforgeeks.org/machine-learning/recursive-least-square-algorithm/
16. Implementation of RLS Algorithm, https://cscan.org/download/?id=788
17. RLS Filter - Compute filtered output, filter error, and filter weights for given input and desired signal using RLS adaptive filter algorithm - Simulink - MathWorks, https://www.mathworks.com/help/dsp/ref/rlsfilter.html
18. gapost/rls: Recursive least squares in C++ - GitHub, https://github.com/gapost/rls
19. Reducing Motion Artifact in Wearable Bio-Sensors Using MEMS Accelerometers for Active Noise Cancellation,

https://skoge.folk.ntnu.no/prost/proceedings/acc05/PDFs/Papers/0285_WeC13_4.pdf
20. Validity of Ultra-short-term HRV, a meta-analysis - OSF, https://osf.io/f8d9b/
21. Driver Stress Detection Using Ultra-Short-Term HRV Analysis under Real World Driving Conditions - MDPI, https://www.mdpi.com/1099-4300/25/2/194
22. Arousal detection by using ultra-short-term heart rate variability (HRV) analysis - Frontiers, https://www.frontiersin.org/journals/medical-engineering/articles/10.3389/fmede.2023.1209252/full
23. Detecting Stress During Real-World Driving Tasks Using Physiological Sensors - CMU School of Computer Science, https://www.cs.cmu.edu/~cga/behavior/Healey.pdf
24. Estimating Markers of Driving Stress through Multimodal Physiological Monitoring This work involved human subjects in its research. Participant consent was obtained and related research was performed in accordance with institutional guidance from Toyota Legal One. Code is available at https://github.com/usc-sail/driving-stressors. - arXiv, https://arxiv.org/html/2507.14146v1
25. Intelligent System for Emotion Recognition Based on Electrodermal Activity Processing, https://www.researchgate.net/publication/265971867_Intelligent_System_for_Emotion_Recognition_Based_on_Electrodermal_Activity_Processing
26. The Micro-Mobility Sensing Gap: A Systematic Review of Physiological Safety Monitoring from Cycling to E-Scooters - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC12944246/
27. A Comprehensive Stacking Ensemble Approach for Stress Level Classification in Higher Education - Bright Publisher, https://bright-journal.org/Journal/index.php/JADS/article/download/388/265
28. DriftSched: Adaptive QoS-Aware Scheduling under Runtime Token Drift for Multi-Tenant GPU Inference - arXiv, https://arxiv.org/html/2606.02982v1
29. Real-Time Outlier Detection in Fast-Moving Data Streams, https://ejournal.gomit.id/ijaaiml/article/download/287/400/7832
30. Smoothing Drone Telemetry in Real Time with Exponential Moving Average Using Python, https://www.c-sharpcorner.com/article/smoothing-drone-telemetry-in-real-time-with-exponential-moving-average-using-pyt/
31. Analyzing the Impact of High-Frequency Noise on Hydrological Runoff Modeling: A Frequency-Based Framework for Data Uncertainty Assessment - MDPI, https://www.mdpi.com/2073-4441/18/2/195
32. Exponential smoothing - Wikipedia, https://en.wikipedia.org/wiki/Exponential_smoothing
33. Detecting Analog Sensor Drift in PLC Using Standard Deviation - Industrial Monitor Direct, https://industrialmonitordirect.com/blogs/knowledgebase/plc-standard-deviation-for-analog-drift-detection-and-predictive-maintenance

34. Building a Robust Classifier with Stacked Generalization - DEV Community, https://dev.to/ddebajyati/building-a-robust-classifier-with-stacked-generalization-5flk
35. StackingClassifier — scikit-learn 1.9.0 documentation, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.StackingClassifier.html
36. The stacking ensemble method | Towards Data Science, https://towardsdatascience.com/the-stacking-ensemble-method-984f5134463a/
37. Stacking Ensemble Machine Learning With Python - MachineLearningMastery.com, https://machinelearningmastery.com/stacking-ensemble-machine-learning-with-python/
38. The Stacking Principle in Machine Learning: Base Learners, Meta-Learners, and the Art of Learning to Learn | by Lumina | Medium, https://medium.com/@ayasc/the-stacking-principle-in-machine-learning-base-learners-meta-learners-and-the-art-of-learning-524a085f3255
39. GitHub - jonnor/embeddedml: Notes on Machine Learning on edge for embedded/sensor/IoT uses, https://github.com/jonnor/embeddedml
40. Enhancing Machine Learning Predictions with Stacking Ensemble Techniques - CodeSignal, https://codesignal.com/learn/courses/ensemble-methods-from-scratch/lessons/enhancing-machine-learning-predictions-with-stacking-ensemble-techniques
41. StackingClassifier: Simple stacking - mlxtend - GitHub Pages, https://rasbt.github.io/mlxtend/user_guide/classifier/StackingClassifier/
42. Deception Detection from Five-Channel Wearable EEG on LieWaves: A Reproducible Baseline for Subject-Dependent and Subject-Independent Evaluation - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC12899830/
43. GroupKFold and Stratified GroupKFold Efficientnet - Kaggle, https://www.kaggle.com/code/reighns/groupkfold-and-stratified-groupkfold-efficientnet
44. GroupKFold — scikit-learn 1.9.0 documentation, https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GroupKFold.html
45. Efficient Feature-Selection-Based Stacking Model for Stress Detection Based on Chest Electrodermal Activity - PMC, https://pmc.ncbi.nlm.nih.gov/articles/PMC10422546/
46. Stacking made easy with Sklearn - Maarten Grootendorst, https://www.maartengrootendorst.com/blog/stacking/
47. Machine learning for microcontroller and embedded systems. Train in Python, then do inference on any device with a C99 compiler. - emlearn documentation - Read the Docs, https://emlearn.readthedocs.io/en/latest/source/README.html